Tries

Search, insertion and deletion in a trie whose leaves keep whole words

A trie is a tree data structure that stores a set of words, where the path from the root to a word is spelled by the word's own letters. The name comes from *retrieval* [1]. The variant studied in this lab uses two kinds of nodes: branch nodes, with one pointer for every letter, and leaves, which keep whole words [2, 3].

Contents

1 Objectives **1**

2 Background **2**

2.1 The nodes of the tree 2

2.2 The TrieNode structure 2

3 Trie operations **3**

3.1 Search 3

3.2 Insertion 3

3.3 Deletion 4

3.4 Listing 4

3.5 Cost of the operations 4

4 Lab work **5**

4.1 The complete functions 5

4.2 Expected results 8

5 Exercises **8**

6 Practice questions **10**

7 Review questions **10**

8 Source code **11**

9 References **16**

1 Objectives

- Understanding how a trie with branch nodes and word-holding leaves is organized.
- Explaining the role of the blank pointer for words that are prefixes of other words.
- Implementing search, insertion, deletion and prefix listing in C++.
- Analysing the cost of the operations in terms of word length, and the memory taken by branch nodes.
- Comparing this variant with the trie that has one node per letter and with other search structures.

2 Background

2.1 The nodes of the tree

A **branch node** corresponds to a prefix *p* shared by at least two stored words. It has 27 pointers: `child[0]` … `child[25]` for the letters a…z and `child[26]`, the **blank pointer**. The pointer `child[c - 'a']` leads to the words that start with *p* followed by the letter *c*, and the blank pointer leads to the word equal to *p* itself, if there is one.

A **leaf** keeps a whole word and has no children. It sits right below the shortest prefix of its word that no other word shares. A leaf can therefore be reached before all the letters of the word have been read, so a search has to compare the whole word at the end.

The root is always a branch node and corresponds to the empty prefix. Every other branch node separates at least two words. For a given set of words the shape of the tree is unique: it does not depend on the order in which the words were inserted. Figure 1 shows the tree used as an example throughout this lab.

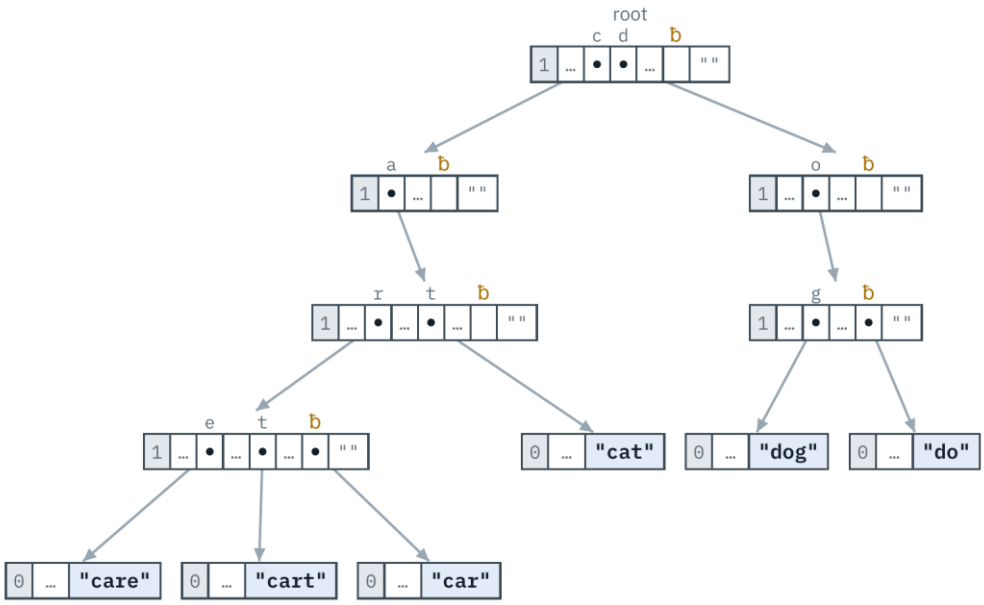


Figure 1. The trie for the words `car`, `cart`, `care`, `cat`, `do`, `dog`. The nodes are drawn as in the course. A branch node has the NotLeaf flag set to 1, one cell for every letter in use, the blank pointer ƀ and an empty word field; the "…" cells hold null pointers. A leaf has NotLeaf set to 0 and keeps the word. The word `car` takes the blank pointer of node `car` because it is a prefix of `cart` and `care`.

2.2 The TrieNode structure

In C++ both kinds of nodes use the same structure, `TrieNode`, described in Table 1.

Table 1. The fields of `TrieNode`.

FIELD	TYPE	ROLE
`notLeaf`	`bool`	`true` (1 in the figures) for a branch node, `false` (0) for a leaf
`child[27]`	`TrieNode*`	the children of a branch node: indices 0…25 for a…z, index 26 for blank; all `nullptr` in a leaf
`word`	`std::string`	the word kept by a leaf; empty in branch nodes

On a 64-bit machine the `child` array takes 27 × 8 = 216 bytes, and a whole node takes 256 bytes with g++ (`sizeof(TrieNode)`). The 27 pointers are allocated in leaves too, where they stay unused.

3 Trie operations

Every operation walks down from the root using the index `slot(w, d)`: the letter `w[d]` while `d` is less than the length of `w`, and the blank pointer after that. A word that has ended thus continues on the blank pointer.

3.1 Search

Start at the root and follow `child[slot(w, d)]` for d = 0, 1, 2, … A null pointer means that no word continues along that path, so `w` is not there. A branch node means going down one level. A leaf ends the search: `w` is present only if the leaf holds exactly `w`.

In Figure 1, searching for `cat` follows the letters c, a, t and reaches the leaf `cat`; the whole-word comparison succeeds. Searching for `dogs` reaches the leaf `dog` after three letters, and the comparison fails. Searching for `ca` reaches node `ca`, whose blank pointer is null: `ca` is only a prefix of other words, so it is not there.

3.2 Insertion

Walk down as in a search. If the pointer followed is null, the leaf with `w` goes right there. If the walk reaches a leaf with the same word, the insertion stops. If the leaf holds another word `v`, it is replaced with a branch node, and one more branch node is added for every position where `w` and `v` have the same letter. At the first position where they differ, the two leaves are attached to their own pointers; if one of the words has ended, its leaf goes on the blank pointer.

When `carts` is inserted into the tree of Figure 1, the path c, a, r, t reaches the leaf `cart`. The two words differ right at the next position: `cart` has ended, while `carts` goes on with s. The leaf is replaced with the branch node `cart`, and below it go the leaf `cart`, on the blank pointer, and the leaf `carts`, on the pointer s (Figure 2).

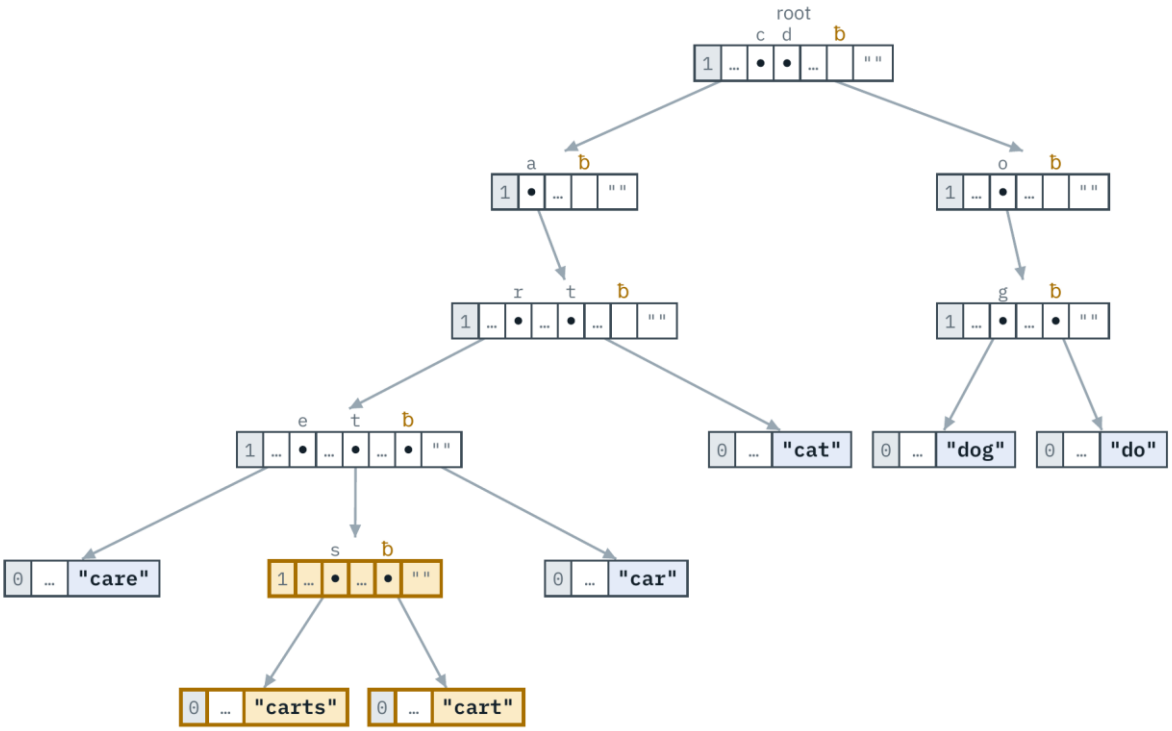


Figure 2. The tree of Figure 1 after inserting `carts`. The highlighted nodes are the ones that changed: the new branch node `cart` and the two leaves attached to it.

3.3 Deletion

Find the leaf of `w`, remembering the branch nodes on the path, and set the pointer to it to null. Then walk back up: a branch node left with a single child that is a leaf no longer separates anything and is replaced with that leaf. The walk stops at a node with at least two children, at a node whose only child is another branch node, or at the root, which is never deleted.

After `dog` is deleted from the tree of Figure 1, node `do` is left with only the leaf `do`, on its blank pointer. The leaf moves up onto the pointer `o` of node `d`, and node `do` is deleted. Node `d` is now left with that single leaf as well, so the leaf moves up again, onto the pointer `d` of the root, and node `d` is deleted. The walk stops at the root (Figure 3).

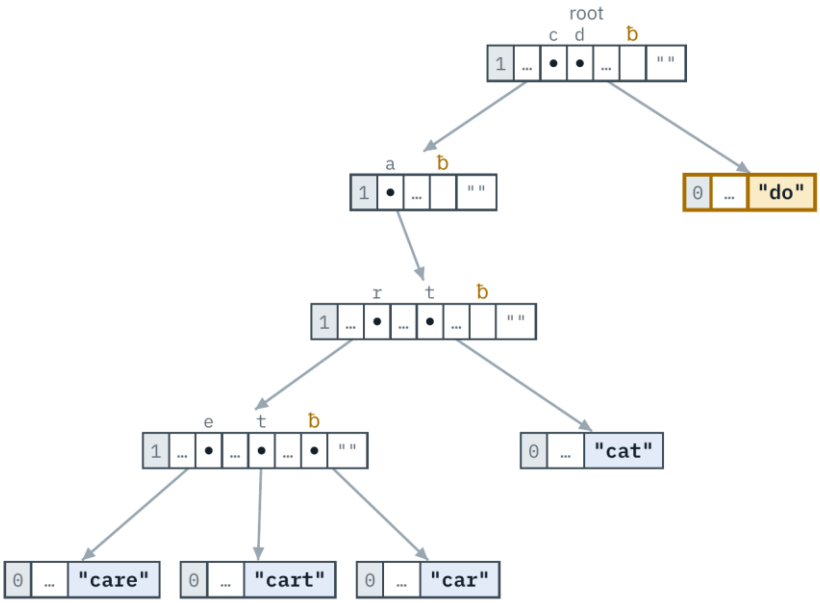


Figure 3. The tree of Figure 1 after deleting `dog`. The highlighted leaf, `do`, has moved up two levels, in place of the deleted nodes `do` and `d`.

3.4 Listing

A preorder traversal that visits the blank pointer first and then the letters a…z produces the words in alphabetical order, with no sorting. For a prefix `p`, walk down along the letters of `p` and list the subtree reached. If a leaf appears on the way, it is the only candidate, and it is checked against `p`. For the prefix `ca`, for example, the walk follows c and a, and the subtree of node `ca` gives `car`, `care`, `cart`, `cat`.

3.5 Cost of the operations

Table 2 sums up the cost of the operations. The cost of a search does not depend on the number of words: a 10-letter word needs at most 11 levels, however large the dictionary.

Table 2. Cost of the operations: *L* is the word length, *|p|* the prefix length, *b* the number of branch nodes in the subtree visited, *k* the total length of the words listed.

OPERATION	COST	WHY
search	`O(L)`	at most L + 1 levels, plus comparing the word in the leaf
insertion	`O(L)`	the walk down, plus at most L new branch nodes when a leaf is split
deletion	`O(27 · L)`	the walk down and back up; on every level `onlyChild` checks 27 pointers
prefix listing	`O(|p| + 27 · b + k)`	the walk along the prefix and the traversal of the subtree
memory	`256 bytes / node`	27 pointers in every node, few of which are usually non-null

4 Lab work

The functions in section 4.1 implement the operations of section 3, and the complete menu-driven program is in section 8. The lab goes as follows:

1. Study the `TrieNode` structure and the functions of the `Trie` class in section 4.1. For every function, follow the nodes it visits on the tree of Figure 1.
2. Build the program from section 8 and create the tree of Figure 1 with option 2 of the menu. Compare the drawing printed by option 7 with Figure 1.
3. Apply the operations of Table 3, in order, and check the results. After every insertion and deletion, draw the tree with option 7 and compare it with Figures 2 and 3.
4. Solve the exercises in section 5 and answer the questions in sections 6 and 7.

4.1 The complete functions

The `TrieNode` structure and the `Trie` class, with all the operations of section 3. The function `slot` gives the index of the pointer followed at depth `d`, and `nth` gives the order in which children are visited: blank first, then a…z.

`struct TrieNode · class Trie`

```
const int ALPHABET = 26;          // the letters 'a'..'z'
const int BLANK = ALPHABET;       // index of the blank pointer
const int NR = ALPHABET + 1;      // 27 pointers in a branch node

struct TrieNode {
    bool notLeaf;                 // true: branch node; false: leaf (holds a word)
    TrieNode* child[NR];          // children of a branch node; stay nullptr in leaves
    std::string word;             // the word held by a leaf; empty in branch nodes

    explicit TrieNode(bool intern, const std::string& w = "")
        : notLeaf(intern), word(w) {
        for (int i = 0; i < NR; i++) child[i] = nullptr;
    }
    ~TrieNode() {
        for (int i = 0; i < NR; i++) delete child[i];   // deletes the whole subtree
    }
};

class Trie {
public:
    Trie() : root(new TrieNode(true)), nWords(0), nBranches(1) {}
    ~Trie() { delete root; }
    Trie(const Trie&) = delete;              // the tree owns its nodes: no copies
    Trie& operator=(const Trie&) = delete;

```

```
    // true if s has only letters 'a'..'z' (an empty string is fine, as a prefix)
    static bool onlyLetters(const std::string& s) {
        for (char c : s)
            if (c < 'a' || c > 'z') return false;
        return true;
    }
    // a valid word is non-empty and holds only lowercase letters
    static bool isWord(const std::string& w) { return !w.empty() && onlyLetters(w); }

    // inserts w; returns false if w is invalid or already present
    bool insert(const std::string& w) {
        if (!isWord(w)) return false;
        TrieNode* node = root;                          // the current branch node
        size_t d = 0;                                   // depth = letters consumed
        for (;;) {
            int i = slot(w, d);
            TrieNode* next = node->child[i];
            if (next == nullptr) {                      // free: the leaf goes here
                node->child[i] = new TrieNode(false, w);
                nWords++;
                return true;
            }
            if (next->notLeaf) {                        // branch node: go down
                node = next;
                d++;
                continue;
            }
            if (next->word == w) return false;          // the word is already there
            // a leaf with another word: separate it from w with branch nodes
            TrieNode* old = next;
            do {
                TrieNode* branch = new TrieNode(true);
                node->child[i] = branch;
                nBranches++;
                node = branch;
                d++;
                i = slot(w, d);
            } while (i == slot(old->word, d));          // same letter: one more level
            node->child[slot(old->word, d)] = old;
            node->child[i] = new TrieNode(false, w);
            nWords++;
            return true;
        }
    }

    // searches for w; a leaf may come before the end of w: compare the whole word
    bool find(const std::string& w) const {
        if (!isWord(w)) return false;
        const TrieNode* node = root;
        for (size_t d = 0; ; d++) {
            const TrieNode* next = node->child[slot(w, d)];
            if (next == nullptr) return false;           // no such branch
            if (!next->notLeaf) return next->word == w;  // leaf: compare the word
            node = next;                                 // branch node: go down
        }
    }

    // deletes w; returns false if w is not present
    bool remove(const std::string& w) {
        if (!isWord(w)) return false;
        std::vector<TrieNode*> path;                    // path[k]: node at depth k
        TrieNode* node = root;
        for (size_t d = 0; ; d++) {
            path.push_back(node);
            TrieNode* next = node->child[slot(w, d)];
            if (next == nullptr) return false;
            if (!next->notLeaf) {
                if (next->word != w) return false;
                node->child[slot(w, d)] = nullptr;      // take the leaf out
                delete next;
                nWords--;
                break;
            }
```

```
            node = next;
        }
        // go up: a branch node left with only a leaf is replaced by that leaf
        for (size_t k = path.size() - 1; k > 0; k--) {
            TrieNode* branch = path[k];
            int only = onlyChild(branch);
            if (only < 0 || branch->child[only]->notLeaf) break;
            TrieNode* leaf = branch->child[only];
            branch->child[only] = nullptr;              // detach the leaf
            path[k - 1]->child[slot(w, k - 1)] = leaf;  // the leaf moves up
            delete branch;
            nBranches--;
        }
        return true;
    }

    // the words that start with p, in alphabetical order (empty p: all words)
    std::vector<std::string> wordsWithPrefix(const std::string& p) const {
        std::vector<std::string> out;
        if (!onlyLetters(p)) return out;
        const TrieNode* node = root;
        for (size_t d = 0; d < p.size(); d++) {
            const TrieNode* next = node->child[p[d] - 'a'];
            if (next == nullptr) return out;            // no word has this prefix
            if (!next->notLeaf) {                       // a leaf on the path of p
                if (next->word.compare(0, p.size(), p) == 0) out.push_back(next->word);
                return out;
            }
            node = next;
        }
        collect(node, out);                             // the subtree has prefix p
        return out;
    }

    // draws the tree; '_' marks the blank pointer
    void print() const {
        std::cout << "(root)\n";
        if (nWords == 0) std::cout << "    the tree is empty\n";
        else draw(root, "");
    }

    int words() const { return nWords; }
    int branches() const { return nBranches; }
    void clear() { delete root; root = new TrieNode(true); nWords = 0; nBranches = 1; }

private:
    TrieNode* root;      // the root is always a branch node
    int nWords;          // number of words = number of leaves
    int nBranches;       // number of branch nodes, the root included

    // the child followed at depth d: letter w[d], or blank once w has ended
    static int slot(const std::string& w, size_t d) {
        return d < w.size() ? w[d] - 'a' : BLANK;
    }

    // children are visited blank first, then 'a'..'z': alphabetical order
    static int nth(int k) { return k == 0 ? BLANK : k - 1; }

    // index of the only child of n, or -1 if n has several children
    static int onlyChild(const TrieNode* n) {
        int found = -1;
        for (int i = 0; i < NR; i++)
            if (n->child[i] != nullptr) {
                if (found != -1) return -1;
                found = i;
            }
        return found;
    }

    // preorder traversal: appends every word in the subtree of n to out
    void collect(const TrieNode* n, std::vector<std::string>& out) const {
        if (!n->notLeaf) { out.push_back(n->word); return; }
        for (int k = 0; k < NR; k++)
```

```
            if (n->child[nth(k)] != nullptr) collect(n->child[nth(k)], out);
    }

    void draw(const TrieNode* n, const std::string& indent) const {
        std::vector<int> kids;
        for (int k = 0; k < NR; k++)
            if (n->child[nth(k)] != nullptr) kids.push_back(nth(k));
        for (size_t j = 0; j < kids.size(); j++) {
            bool last = (j + 1 == kids.size());
            const TrieNode* c = n->child[kids[j]];
            char label = (kids[j] == BLANK) ? '_' : char('a' + kids[j]);
            std::cout << indent << (last ? "`-- " : "+-- ") << label;
            if (!c->notLeaf) std::cout << " : " << c->word << '\n';
            else {
                std::cout << '\n';
                draw(c, indent + (last ? "    " : "|   "));
            }
        }
    }
};
```

4.2 Expected results

Table 3 lists the results of the operations applied, in order, to the tree of Figure 1. The column "Branch" gives the number of branch nodes after the operation, the root included; the program prints it in the menu header.

Table 3. Results of the operations applied, in order, to the tree of Figure 1.

OPERATION	RESULT	BRANCH	EXPLANATION
`t.find("care")`	`true`	6	the leaf `care`, at depth 4
`t.find("ca")`	`false`	6	node `ca` has a null blank pointer
`t.find("cow")`	`false`	6	null pointer for `o` in node `c`
`t.find("dogs")`	`false`	6	the path `dog` leads to the leaf `dog` ≠ `dogs`
`t.wordsWithPrefix("ca")`	`{car, care, cart, cat}`	6	the subtree of node `ca`, traversed in preorder
`t.wordsWithPrefix("cart")`	`{cart}`	6	the leaf `cart`, reached on the path of the prefix, starts with the prefix
`t.wordsWithPrefix("x")`	`{}`	6	null pointer for `x` in the root
`t.insert("carts")`	`true`	7	the leaf `cart` is split: 1 new node, `cart`
`t.remove("care")`	`true`	7	node `car` still has 2 children and stays
`t.remove("dog")`	`true`	5	nodes `do`, `d` disappear; the leaf `do` moves up
`t.insert("c")`	`true`	5	a new leaf on the blank pointer of node `c`

5 Exercises

1. **Statistics.** Implement `int height()`, the largest depth of a leaf, and `int leafDepth(const std::string& w)`. Count the non-null pointers and print their ratio to 27 × the number of branch nodes, for a dictionary of at least 10^4 words.
2. **A trie with one node per letter.** Implement the classic variant, where every letter has its own node and the end of a word is marked with `bool isEnd`. For the same set of words, compare the number of nodes and the memory used with the variant in this lab.

3. **Counting the words with a prefix.** Add to every branch node a counter of the words in its subtree, kept up to date by insertion and deletion, so that `int countPrefix(const std::string& p)` runs in O(|p|) without traversing the subtree.
4. **Autocomplete.** Implement `std::vector<std::string> complete(const std::string& p, size_t k)`, which returns the first `k` words with prefix `p` in alphabetical order and stops the traversal as soon as it has them.
5. **Longest common prefix.** Implement `std::string longestCommonPrefix()` for all the words in the tree. Walk down from the root while the current node has a single child that is a branch node, and handle reaching a leaf or the blank pointer separately.
6. **Accented letters.** Extend the alphabet with the Romanian letters ă, â, î, ș, ț (31 letters plus blank). Words are read as UTF-8, so an accented letter takes two bytes: first turn the word into a sequence of indices 0…30.
7. **Bonus: compressed trie.** In the variant of this lab, every branch node corresponds to a single prefix, one letter longer than the prefix of its parent. Before the place where words part, there can therefore be nodes with a single child, which separate nothing and only pass the letters on. In Figure 4a, for the words `cart`, `carton` and `cartoon`, the nodes `c`, `ca` and `car` have a single child each. A compressed trie removes these nodes: every chain of single-child branch nodes is replaced with one edge, labelled with all the letters of the chain. In Figure 4b the edge `cart` leads straight from the root to the node where `cart` and `cartoon` part, and the tree has 3 branch nodes instead of 6. Implement this variant, keeping whole words in the leaves:

   a) every branch node also keeps, for each child, the string on the edge to it, for example in an array `std::string label[27]`;
   b) a search picks the child by the current letter, as before, then compares the rest of the edge string with the next letters of the word; at the first difference, the word is not there;
   c) an insertion that parts from the edge string halfway through splits the edge in two and creates a new branch node at the split point; for example, inserting `case` into Figure 4b splits the edge `cart` into `ca` and `rt`;
   d) a deletion that leaves a branch node with a single child that is also a branch node merges the two, joining the strings on their edges; if the only child is a leaf, the leaf moves up, as in this lab.

   Compare the number of branch nodes in the two variants for a dictionary of at least 10^4 words.

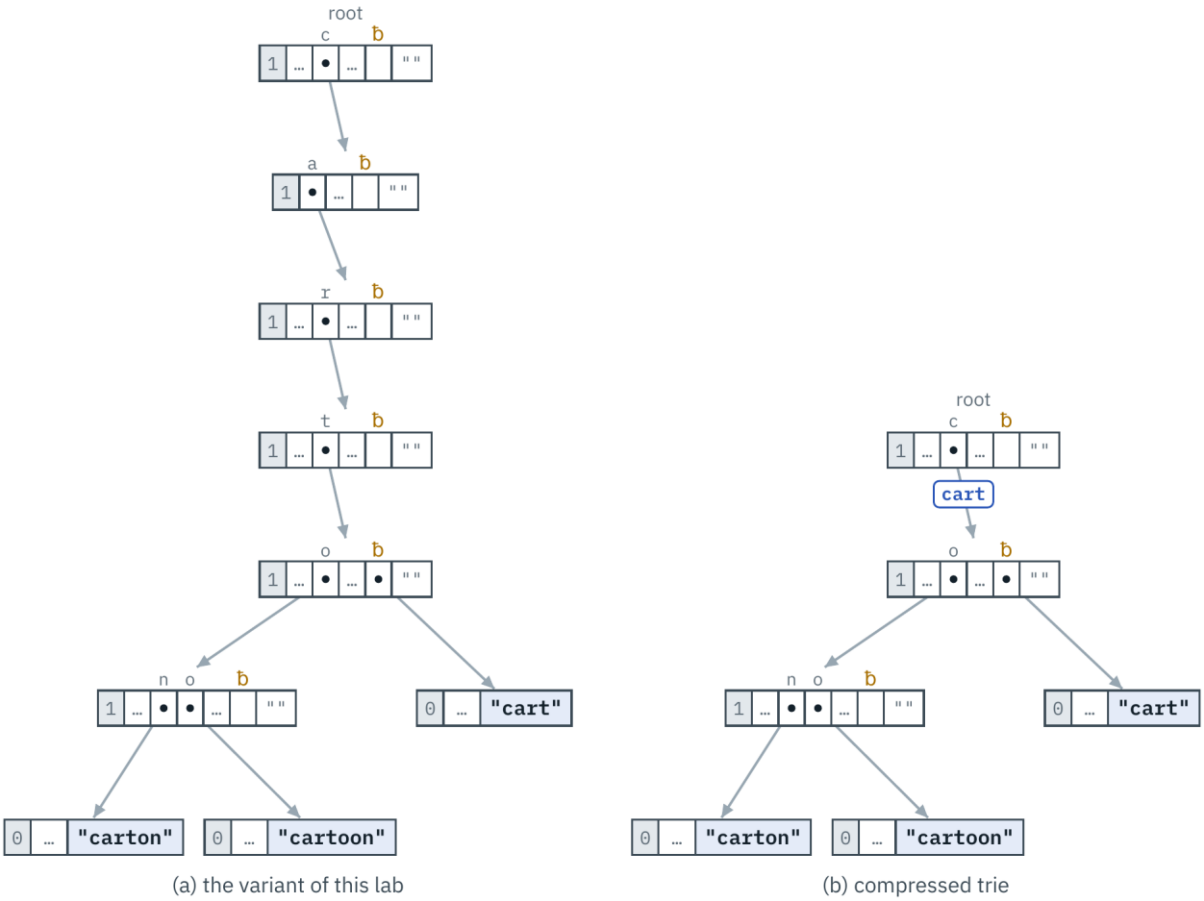


Figure 4. The words `cart`, `carton` and `cartoon`: (a) in the variant of this lab, with 6 branch nodes; (b) in a compressed trie, with 3 branch nodes. The chain of nodes `c`, `ca`, `car`, each with a single child, becomes the edge `cart`.

6 Practice questions

Answer the questions on paper, drawing the tree where it helps.

1. Insert the words `sea`, `seal`, `seat`, `ten`, `tend`, `te`, `se`, in this order, into an empty trie. Draw the tree after every insertion. How many branch nodes does the final tree have, and at what depth is every leaf?
2. On the final tree of question 1, search for `tent`, `s` and `seats`. Where does each search stop, and what is compared?
3. From the same tree, delete `tend` and then `te`. Which branch nodes disappear after each deletion, and where does the leaf `ten` end up?
4. On the tree of question 1, what does `wordsWithPrefix` return for the prefixes `se`, `tend` and `tea`?
5. What is the largest number of branch nodes in a trie with two words of at most 5 letters? Give an example of such words.
6. Which order of the words of question 1 does a traversal give if it visits the letters a…z before the blank pointer? Why is it not alphabetical?

7 Review questions

1. Why does a search compare the whole word when it reaches a leaf, even if all the letters read so far match?

2. What is the blank pointer for? Which words could not be stored without it?
3. Why does a traversal that visits the blank pointer before the letters a...z produce the words in alphabetical order?
4. Why, in a deletion, is a branch node left with a single child removed only when that child is a leaf?
5. Why does the shape of the tree not depend on the order in which the words are inserted?
6. What does it cost to search for a word of length L in a trie with n words? Compare with a balanced binary search tree and with a hash table.
7. How much memory does a node take on a 64-bit machine? When does this variant become wasteful, and how could the memory be reduced?

8 Source code

The complete menu-driven program starts from the implementation in the course's ADS repository [5] and keeps its node structure: the NotLeaf flag, the 27 pointers with index 26 for blank, and the word stored in the leaf. The code is rewritten in C++ and extended as follows:

- the `Trie` class owns the nodes and frees them in its destructor; leaves use `std::string`, so words have no maximum length;
- insertion, search and listing follow the `slot` function, which treats the end of a word as one more letter, blank;
- deletion is added, with the replacement of nodes left with a single leaf, along with alphabetical listing, prefix search and a drawing of the tree;
- words are validated, capital letters are turned into lowercase, and the 8-option menu runs on any system, with no `conio.h`.

Build: `g++ -std=c++11 -Wall -Wextra -o trie_menu trie_menu_en.cpp`.

`trie_menu_en.cpp`

```cpp
// trie_menu_en.cpp - Tries, Data Structures and Algorithms lab
//
// The variant with branch nodes and leaves:
//  - a branch node has 27 pointers: one for each of 'a'..'z' plus the blank pointer
//    (index 26), used by the word that ends exactly at that node;
//  - every word is kept whole in a leaf, placed as high in the tree as possible:
//    right below the first prefix that no other word shares.
//
// Build: g++ -std=c++11 -Wall -Wextra -o trie_menu trie_menu_en.cpp

#include <iostream>
#include <sstream>
#include <string>
#include <vector>
#include <cctype>

const int ALPHABET = 26;          // the letters 'a'..'z'
const int BLANK = ALPHABET;       // index of the blank pointer
const int NR = ALPHABET + 1;      // 27 pointers in a branch node

struct TrieNode {
    bool notLeaf;                 // true: branch node; false: leaf (holds a word)
    TrieNode* child[NR];          // children of a branch node; stay nullptr in leaves
    std::string word;             // the word held by a leaf; empty in branch nodes

    explicit TrieNode(bool intern, const std::string& w = "")
        : notLeaf(intern), word(w) {
        for (int i = 0; i < NR; i++) child[i] = nullptr;
    }
    ~TrieNode() {
        for (int i = 0; i < NR; i++) delete child[i];   // deletes the whole subtree
```

```
    }
};

class Trie {
public:
    Trie() : root(new TrieNode(true)), nWords(0), nBranches(1) {}
    ~Trie() { delete root; }
    Trie(const Trie&) = delete;              // the tree owns its nodes: no copies
    Trie& operator=(const Trie&) = delete;

    // true if s has only letters 'a'..'z' (an empty string is fine, as a prefix)
    static bool onlyLetters(const std::string& s) {
        for (char c : s)
            if (c < 'a' || c > 'z') return false;
        return true;
    }
    // a valid word is non-empty and holds only lowercase letters
    static bool isWord(const std::string& w) { return !w.empty() && onlyLetters(w); }

    // inserts w; returns false if w is invalid or already present
    bool insert(const std::string& w) {
        if (!isWord(w)) return false;
        TrieNode* node = root;                          // the current branch node
        size_t d = 0;                                   // depth = letters consumed
        for (;;) {
            int i = slot(w, d);
            TrieNode* next = node->child[i];
            if (next == nullptr) {                      // free: the leaf goes here
                node->child[i] = new TrieNode(false, w);
                nWords++;
                return true;
            }
            if (next->notLeaf) {                        // branch node: go down
                node = next;
                d++;
                continue;
            }
            if (next->word == w) return false;          // the word is already there
            // a leaf with another word: separate it from w with branch nodes
            TrieNode* old = next;
            do {
                TrieNode* branch = new TrieNode(true);
                node->child[i] = branch;
                nBranches++;
                node = branch;
                d++;
                i = slot(w, d);
            } while (i == slot(old->word, d));          // same letter: one more level
            node->child[slot(old->word, d)] = old;
            node->child[i] = new TrieNode(false, w);
            nWords++;
            return true;
        }
    }

    // searches for w; a leaf may come before the end of w: compare the whole word
    bool find(const std::string& w) const {
        if (!isWord(w)) return false;
        const TrieNode* node = root;
        for (size_t d = 0; ; d++) {
            const TrieNode* next = node->child[slot(w, d)];
            if (next == nullptr) return false;           // no such branch
            if (!next->notLeaf) return next->word == w;  // leaf: compare the word
            node = next;                                 // branch node: go down
        }
    }

    // deletes w; returns false if w is not present
    bool remove(const std::string& w) {
        if (!isWord(w)) return false;
        std::vector<TrieNode*> path;                    // path[k]: node at depth k
        TrieNode* node = root;
        for (size_t d = 0; ; d++) {
```

```
            path.push_back(node);
            TrieNode* next = node->child[slot(w, d)];
            if (next == nullptr) return false;
            if (!next->notLeaf) {
                if (next->word != w) return false;
                node->child[slot(w, d)] = nullptr;      // take the leaf out
                delete next;
                nWords--;
                break;
            }
            node = next;
        }
        // go up: a branch node left with only a leaf is replaced by that leaf
        for (size_t k = path.size() - 1; k > 0; k--) {
            TrieNode* branch = path[k];
            int only = onlyChild(branch);
            if (only < 0 || branch->child[only]->notLeaf) break;
            TrieNode* leaf = branch->child[only];
            branch->child[only] = nullptr;              // detach the leaf
            path[k - 1]->child[slot(w, k - 1)] = leaf;  // the leaf moves up
            delete branch;
            nBranches--;
        }
        return true;
    }

    // the words that start with p, in alphabetical order (empty p: all words)
    std::vector<std::string> wordsWithPrefix(const std::string& p) const {
        std::vector<std::string> out;
        if (!onlyLetters(p)) return out;
        const TrieNode* node = root;
        for (size_t d = 0; d < p.size(); d++) {
            const TrieNode* next = node->child[p[d] - 'a'];
            if (next == nullptr) return out;            // no word has this prefix
            if (!next->notLeaf) {                       // a leaf on the path of p
                if (next->word.compare(0, p.size(), p) == 0) out.push_back(next->word);
                return out;
            }
            node = next;
        }
        collect(node, out);                             // the subtree has prefix p
        return out;
    }

    // draws the tree; '_' marks the blank pointer
    void print() const {
        std::cout << "(root)\n";
        if (nWords == 0) std::cout << "    the tree is empty\n";
        else draw(root, "");
    }

    int words() const { return nWords; }
    int branches() const { return nBranches; }
    void clear() { delete root; root = new TrieNode(true); nWords = 0; nBranches = 1; }

private:
    TrieNode* root;      // the root is always a branch node
    int nWords;          // number of words = number of leaves
    int nBranches;       // number of branch nodes, the root included

    // the child followed at depth d: letter w[d], or blank once w has ended
    static int slot(const std::string& w, size_t d) {
        return d < w.size() ? w[d] - 'a' : BLANK;
    }

    // children are visited blank first, then 'a'..'z': alphabetical order
    static int nth(int k) { return k == 0 ? BLANK : k - 1; }

    // index of the only child of n, or -1 if n has several children
    static int onlyChild(const TrieNode* n) {
        int found = -1;
        for (int i = 0; i < NR; i++)
            if (n->child[i] != nullptr) {
```

```
                if (found != -1) return -1;
                found = i;
            }
        return found;
    }

    // preorder traversal: appends every word in the subtree of n to out
    void collect(const TrieNode* n, std::vector<std::string>& out) const {
        if (!n->notLeaf) { out.push_back(n->word); return; }
        for (int k = 0; k < NR; k++)
            if (n->child[nth(k)] != nullptr) collect(n->child[nth(k)], out);
    }

    void draw(const TrieNode* n, const std::string& indent) const {
        std::vector<int> kids;
        for (int k = 0; k < NR; k++)
            if (n->child[nth(k)] != nullptr) kids.push_back(nth(k));
        for (size_t j = 0; j < kids.size(); j++) {
            bool last = (j + 1 == kids.size());
            const TrieNode* c = n->child[kids[j]];
            char label = (kids[j] == BLANK) ? '_' : char('a' + kids[j]);
            std::cout << indent << (last ? "`-- " : "+-- ") << label;
            if (!c->notLeaf) std::cout << " : " << c->word << '\n';
            else {
                std::cout << '\n';
                draw(c, indent + (last ? "    " : "|   "));
            }
        }
    }
};

// ---------------------------------------------------------------- menu

// reads a line; returns false at the end of the input
static bool readLine(const std::string& prompt, std::string& line) {
    std::cout << prompt;
    return static_cast<bool>(std::getline(std::cin, line));
}

// "1 word", "5 words"
static std::string plural(int n, const std::string& one, const std::string& many) {
    return std::to_string(n) + ' ' + (n == 1 ? one : many);
}

// lowercase letters, no surrounding spaces; capital letters are accepted
static std::string normalize(const std::string& s) {
    std::string r;
    for (char c : s)
        if (!std::isspace(static_cast<unsigned char>(c)))
            r += static_cast<char>(std::tolower(static_cast<unsigned char>(c)));
    return r;
}

static void printList(const std::vector<std::string>& v) {
    if (v.empty()) { std::cout << "  (no words)\n"; return; }
    std::cout << " ";
    for (const std::string& w : v) std::cout << ' ' << w;
    int n = static_cast<int>(v.size());
    std::cout << "\n  (" << plural(n, "word", "words") << ")\n";
}

int main() {
    Trie t;
    std::string line;
    std::cout << "Tries - DSA lab\n"
                 "Words use only the letters a-z "
                 "(capital letters are turned into lowercase).\n";
    for (;;) {
        std::cout << "\n=== trie | " << plural(t.words(), "word", "words") << " | "
                  << plural(t.branches(), "branch node", "branch nodes")
                  << " ===\n"
                  << " 1. Insert a word\n"
                  << " 2. Insert several words (on one line)\n"
```

```
                  << " 3. Search for a word\n"
                  << " 4. Delete a word\n"
                  << " 5. List the words in alphabetical order\n"
                  << " 6. List the words that start with a prefix\n"
                  << " 7. Draw the tree\n"
                  << " 8. Clear the tree\n"
                  << " 0. Exit\n";
        if (!readLine("Option: ", line)) break;
        std::string opt = normalize(line);
        if (opt == "0") break;

        if (opt == "1" || opt == "3" || opt == "4") {
            if (!readLine("Word: ", line)) break;
            std::string w = normalize(line);
            if (!Trie::isWord(w)) {
                std::cout << "Invalid word: use only the letters a-z.\n";
                continue;
            }
            if (opt == "1") {
                if (t.insert(w)) {
                    std::cout << "Inserted \"" << w << "\".\n";
                    t.print();
                } else {
                    std::cout << "\"" << w << "\" is already in the tree.\n";
                }
            } else if (opt == "3") {
                std::cout << (t.find(w) ? "Found: \"" : "Not found: \"") << w << "\"\n";
            } else {
                if (t.remove(w)) {
                    std::cout << "Deleted \"" << w << "\".\n";
                    t.print();
                } else {
                    std::cout << "\"" << w << "\" is not in the tree.\n";
                }
            }
        } else if (opt == "2") {
            const char* ask = "Words separated by spaces (e.g. car cart care cat): ";
            if (!readLine(ask, line)) break;
            for (char& c : line) if (c == ',' || c == ';') c = ' ';
            std::istringstream in(line);
            std::string token;
            int added = 0;
            while (in >> token) {
                std::string w = normalize(token);
                if (!Trie::isWord(w))
                    std::cout << "  skipped (invalid): " << token << '\n';
                else if (t.insert(w))
                    added++;
                else
                    std::cout << "  already there: " << w << '\n';
            }
            std::cout << "Inserted " << plural(added, "word", "words") << ".\n";
            t.print();
        } else if (opt == "5") {
            printList(t.wordsWithPrefix(""));
        } else if (opt == "6") {
            if (!readLine("Prefix: ", line)) break;
            std::string p = normalize(line);
            if (!Trie::onlyLetters(p)) {
                std::cout << "Invalid prefix: use only the letters a-z.\n";
                continue;
            }
            printList(t.wordsWithPrefix(p));
        } else if (opt == "7") {
            t.print();
        } else if (opt == "8") {
            t.clear();
            std::cout << "The tree is now empty.\n";
        } else {
            std::cout << "Unknown option. Choose a number from 0 to 8.\n";
        }
    }
    return 0;
```

```
324 }
```

9 References

[1] E. Fredkin, “Trie memory”, *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.

[2] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998, section 6.3, “Digital Searching”.

[3] E. Horowitz, S. Sahni, D. Mehta, *Fundamentals of Data Structures in C++*, 2nd ed., Silicon Press, 2006, chapter 12, “Digital Search Structures”.

[4] R. Sedgewick, K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011, section 5.2, “Tries”.

[5] The ADS repository, `Trees/TRIE Trees/trie.cpp`, https://github.com/mihaescu/ADS.